



Linux Driver Development & Examples

ECE 391 – Computer Systems Engineering

Dong Kai Wang

Department of Electrical and Computer Engineering

University of Illinois at Urbana-Champaign

- **Recap of Driver Abstractions**

- Kernel modules, block and character devices, registration, `init` and `exit` functions.

- **Kernel View of Modules**

- The `init_module` system call.
- Registration: major and minor numbers.

- **File Operations Struct**

- Implementing `read/write/open/close` functions.
- Using kernel features and functions.
- Driver security implications.

- **Driver Example: `scull`**

- Using `mknod` and `udev`.
- Implementation of `scull` functions: `read()` and `write()`.

- Download [exampledrivers.tar.xz](#) from ECE 391 course website.
- Compile each driver with make in its directory.
- Should work with modern kernels (tested on Kernel 5.15, Ubuntu 20.04 VM).
 - May want to disable safe boot if running on real hardware.

- Writing basic driver code is slightly different from user applications.
 - Rather than `main()`, should specify `module_init` and `exit` function
 - Makefile should automatically uses `kbuild`.
 - Use `obj-m`, `modules`, `modules_install`, etc. symbols in Makefile.
 - Outputs a kernel object **.ko** file which contain the executable with additional metadata:
 - Module authors, description, license, versions, etc.
 - Use `insmod` / `rmmod` to dynamically load or unload the module.
 - Do not need to recompile the entire kernel and reboot.
- The `init_module` system call.
 - Performs some checks on kernel object.
 - Loads the executable to kernel space, dynamically resolving symbols with kernel.

- Recall that loading a kernel module **does not mean a device is registered**.
 - Your `init` function should register device(s), if any, associated with your driver.
 - Also unregister on exit.
 - So that user programs can interface with your driver device (as a **file**) through system calls.
- Major and minor numbers.
 - Major and minor number combined should uniquely identify a character device.
 - **Major**: corresponds to device type or driver.
 - **Minor**: corresponds to device instance (can have multiple of the same device).
 - *Note*: major numbers are separate between character and block devices (see Piazza note).
- Device registration with `register_chrdev`.
 - `register_chrdev(major_number, "readsimple_dev", &readsimple_fops);`
 - Request a specific major number or let kernel choose a free one for you by requesting 0.

- The `file_operations` struct contains many members, here are some:

`struct module *owner`

- Not an operation, but a pointer to the driver module that owns this struct.
- Prevent it from being unloaded when an operation is taking place.
- Set it using the macro `THIS_MODULE` defined in `<linux/module.h>`.

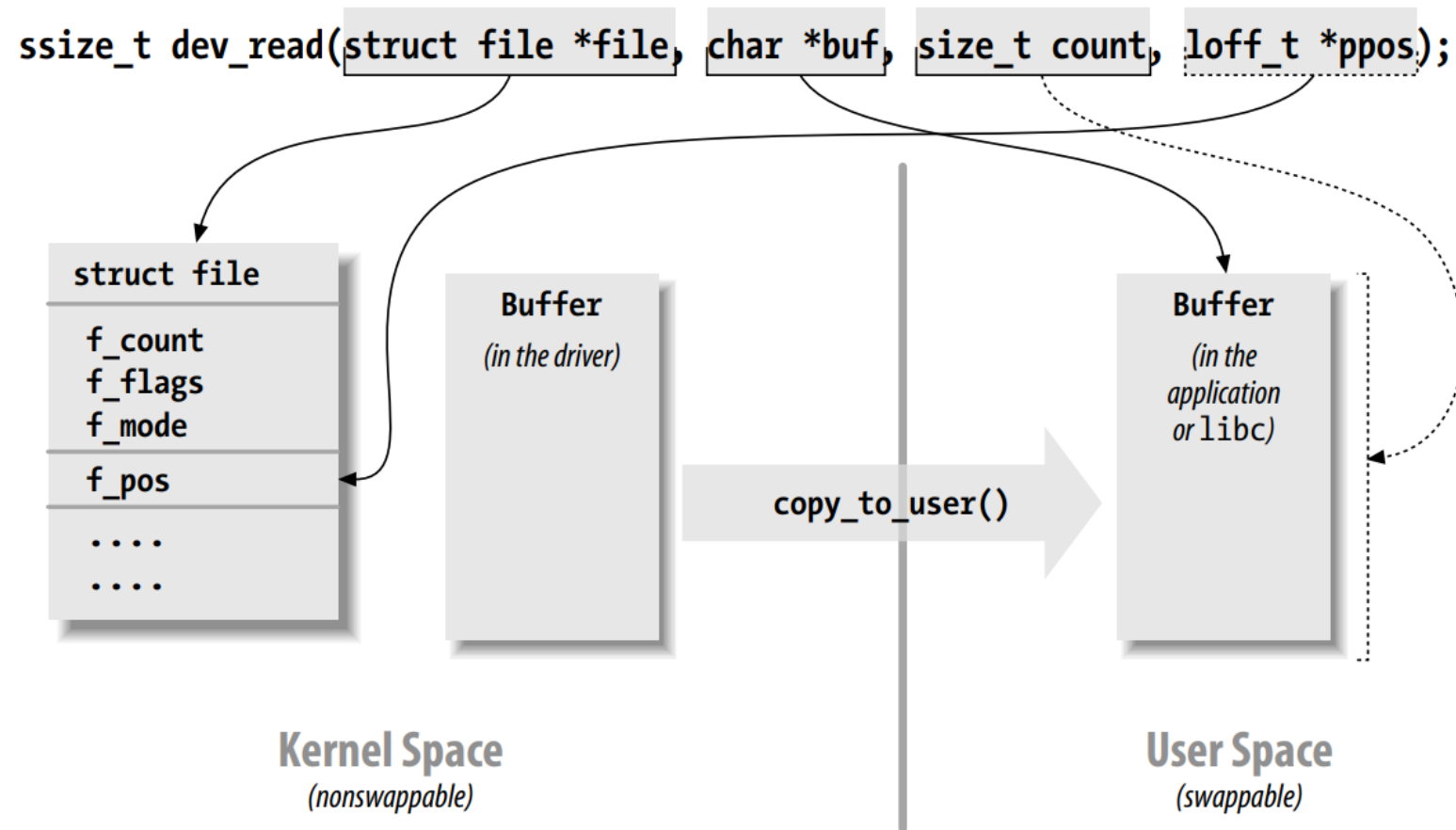
`ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);`

- Return number of bytes successfully read to user (`char __user *`).
- Read system call fails with `-EINVAL` if not implemented (passed NULL function pointer).

`ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);`

- Return number of bytes successfully written to device.
- Write system call fails with `-EINVAL` if not implemented.

- Note that the return value of read may be smaller than count, may retry until completion.



Read Simple Example



```
int  readsimple_major = 0; // Major Number
char readsimple_msg[] = "A Message Example!\n"; // Message Buffer

// Read Function // Pointer to memory in user space, cannot directly dereference this.
ssize_t readsimple_read(struct file *filp, char __user *buf, size_t length, loff_t *pos) {
    int ret = 0;
    if (*pos >= 20) { return 0; } // Already past the end of file
    printk(KERN_ALERT "Attempting read with length = %d \n", (int) length);
    if (length > 20) { length = 20; } // Do not read more than buffer length (20 bytes)
    ret = copy_to_user(buf, readsimple_msg, length);
    printk(KERN_ALERT "Return value of copy_to_user = %d \n", ret);
    if (ret != 0) return -1;
    else { *pos += length; return length; }
}
```


Read Simple Example



```
struct file_operations readsimple_fops = {  
    .read = readsimple_read,  
    .owner = THIS_MODULE  
};
```

File operations struct containing function pointers to this driver's implementation of open/read/write/close/etc.

```
// Init Function
```

```
static int readsimple_init(void)  
{
```

```
    // Register major number, and accept a dynamic number  
    readsimple_major = register_chrdev(0, "readsimple_dev", &readsimple_fops);
```

Register major number for device readsimple_dev.

```
    // Check major number was registered  
    if (readsimple_major < 0) {  
        printk(KERN_ALERT "Failed to Register Char Device readsimple_dev (Major = %d)\n",  
readsimple_major);  
        return readsimple_major;  
    }  
  
    printk(KERN_WARNING "Loaded Read Simple Module with Major Number = %d\n", readsimple_major);  
    return 0;  
}
```

- Use mknod to create a device file in /dev/readsimple_dev that we can access.

```
$ sudo mknod /dev/readsimple_dev c $major 0
```

- Need to also change permissions with chmod for user.
 - You can insmod and rmmod while file is active.
- Check /proc/devices to determine the major number.
- Double check module is loaded with `lsmod`, and device is registered with `ls -l`.
- Modern usage: let udev handle this instead of manually creating a file.
- Note that read/write system calls are not necessarily called **once**.
 - Looks at return value (bytes read / written) and your file offset.
 - Call multiple times until full read or write is fulfilled.
 - Can make your function implementation easier.
 - See example with cat.

```
char readwrite_msg[64];

ssize_t readwrite_write(struct file *filp, char __user *buf, size_t length, loff_t *pos) {
    ssize_t bytes_written = 0;
    if (*pos >= 64)
        return -ENOSPC;
    if (copy_from_user(readwrite_msg + *pos, buf, length))
        return -EFAULT;
    *pos += length;
    bytes_written = length;
    printk(KERN_INFO "Wrote %zu bytes to the device\n", bytes_written);
    return bytes_written;
}
```

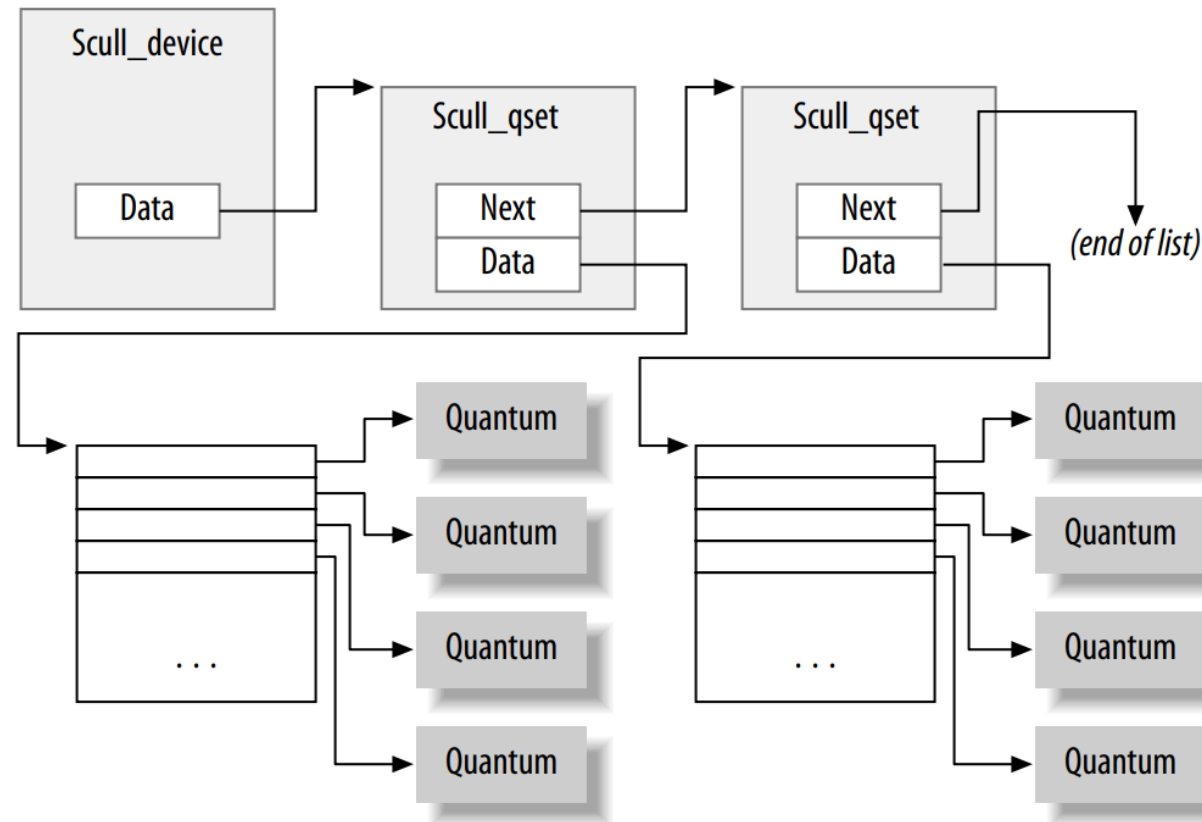
What's wrong here? (Hint: did not check length!)

- Recall that drivers run in **kernel space**.
 - Must be very careful when implementing your functions.
- Have adequate boundary checks:
 - Do not want to copy unintended kernel memory to user or vice versa.
 - Can crash your system or leak sensitive data.
 - Can lead to malicious data / code injection.
- Also, do not load drivers from unknown sources!

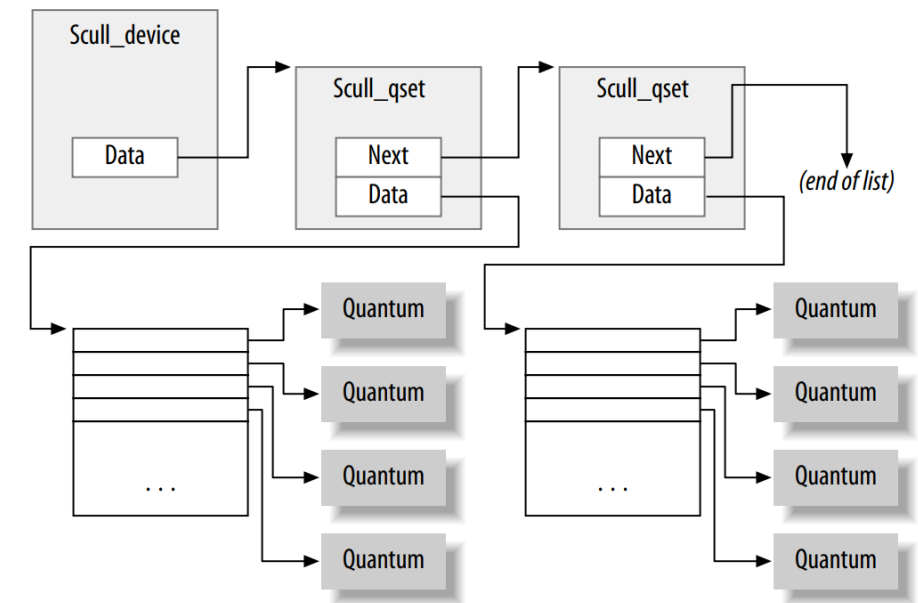
- Example driver: `faulty.c`
 - Dereference a NULL pointer.
 - Read beyond buffer size.
 - (Note: may not compile with gcc on modern OSes due to advanced checks in place).
- Depending on severity can crash or generate a Kernel panic or ***oops***.

```
ssize_t faulty_write (struct file *filp, const char __user *buf, size_t count,
                      loff_t *pos)
{
    /* make a simple fault by dereferencing a NULL pointer */
    *(int *)0 = 0;
    return 0;
}
```

- Simple Character Utility for Loading Localities.



- **S**imple **C**haracter **U**tility for **L**oading **L**ocalities.
- Dynamically allocates a linked list of “Quantum sets” in kernel memory.
 - Purely in memory.
 - Designed to be portable across OS, so pretend these are actually hardware.
 - Each Quantum set contains an array of Quantums, storing some data.
 - Each Quantum set contains 1000 pointers.
 - Each Quantum stores 4000 bytes.
- Showcases various features of drivers.
 - Memory allocation.
 - Race conditions.
 - Privileges and security.
 - And more ...



```
int (*open) (struct inode *, struct file *);
```

- Opens a file, parameters (inode) provided by kernel, not necessary to implement.

```
int (*release) (struct inode *, struct file *);\
```

- Releases (last process closes) the file, also optional.

```
int (*ioctl) (struct inode *, struct file *, unsigned int, unsigned long);
```

- Issue device-specific commands (up to your implementation).
- E.g., can serve as alternative for module parameters.

```
loff_t (*llseek) (struct file *, loff_t, int);
```

- Change current read/write position within a file. The parameter long offset is at least 64 bits.

Read in the scull Driver



```
ssize_t scull_read(struct file *filp, char __user *buf, size_t count,
                  loff_t *f_pos)
{
    struct scull_dev *dev = filp->private_data;
    struct scull_qset *dptr;    /* the first listitem */
    int quantum = dev->quantum, qset = dev->qset;
    int itemsize = quantum * qset; /* how many bytes in the listitem */
    int item, s_pos, q_pos, rest;
    ssize_t retval = 0;
```

```
    if (down_interruptible(&dev->sem))
        return -ERESTARTSYS;
    if (*f_pos >= dev->size)
        goto out;
    if (*f_pos + count > dev->size)
        count = dev->size - *f_pos;
```

Boundary checks and
concurrency with
Semaphores (next lecture)

```
    /* find listitem, qset index, and offset in the quantum */
    item = (long)*f_pos / itemsize;
    rest = (long)*f_pos % itemsize;
    s_pos = rest / quantum; q_pos = rest % quantum;
```

Quantum set indexing
based on single f_pos
parameter.

Read in the scull Driver



```
/* follow the list up to the right position (defined elsewhere) */
dptr = scull_follow(dev, item);
```

```
if (dptr == NULL || !dptr->data || ! dptr->data[s_pos])
    goto out; /* don't fill holes */
```

```
/* read only up to the end of this quantum */
if (count > quantum - q_pos)
    count = quantum - q_pos;
```

Only perform a partial read,
more read calls will come.

```
if (copy_to_user(buf, dptr->data[s_pos] + q_pos, count)) {
    retval = -EFAULT;
    goto out;
}
*f_pos += count;
retval = count;
```

Copy to user, set bytes read
and update file position.

```
out:
    up(&dev->sem);
    return retval;
```

```
}
```



The Grainger College of Engineering

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN